PSC-ONE

統合仕様書

FPGA RISC-V SoC / SynapEngine / PSC-OS

文書版	Rev.0.9 — Preliminary（実装に基づく技術資料）
対象構成	Tang Nano 20K / 安定版 CPU v1 / **標準 NPU legacy**
作成基準日	2026 年 10 月 10 日
プロジェクト	QPSC-Design / PSC-ONE



図 1: PSC-ONE 試作基板。Tang Nano 20K、LCD、マイク、スイッチ等を搭載。

ハードウェアからアプリケーションまで

PSC-ONE は、独自 RISC-V CPU、仮想記憶、キャッシュ、SDRAM、周辺回路、行列演算アクセラレータ、専用 OS を一体で開発するオープンプラットフォームである。CPU やメモリの設計学習、OS 実験、組込み推論、ロボット制御の研究を対象とする。

CPU	32-bit 整数処理、乗除算、CSR、M/S/U、Sv32 経路、選択的 PULP 命令
メモリ	64 Mbit（8 MiB）SDRAM、独立 I/D キャッシュ、32-bit AXI 経路
標準 NPU	legacy：論理 4×4 PE、共有乗算器 4 基（RTL 既定）、INT8 入力／INT32 出力
I/O	UART、SD SPI、LCD SPI、I2S 入力、timer、LED、PIO、I2C slave
ソフトウェア	PSC-OS、FAT32、外部 ELF、MicroPython、JPEG、限定 INT8 FC 推論

本書は指定されたローカルソースと図版を統合した構成仕様であり、半導体製品の電気定格保証書ではない。実装値、過去の測定記録、派生・実験機能を区別する。

目次

1 システム概要と構成境界 **3**

2 CPU アーキテクチャ **4**

3 命令セット、CSR、例外 **5**

4 対応 PULP 命令の詳細仕様と使用例 **6**

5 Sv32 MMU とメモリ保護の範囲 **7**

6 キャッシュ、SDRAM、DMA **8**

7 メモリ配置と MMIO アドレス一覧 **9**

8 UART、タイマー、GPIO、I2C **10**

9 SD カード、FAT32、ブート **11**

10 表示、JPEG、I2S オーディオ **12**

11 標準 NPU：SynapEngine legacy **13**

12 NPU 制御 CSR とソフトウェア契約 **14**

13 派生アクセラレータと実験機能 **15**

14 PSC-OS の構造とタスク実行 **16**

15 ECALL ABI と外部 ELF **17**

16 MicroPython と INT8 推論ランタイム **18**

17 ボード端子、実装条件、ビルド **19**

18 検証範囲、性能値、再現手順 **20**

19 出典、構成差分、文書管理 **21**

本書の読み方

標準構成は CPU v1 と NPU legacy とする。CPU legacy、CPU v2、NPU v1/v2、LPU は比較・拡張のために併記する。CPU と NPU の版は独立に選択する。本文のアドレスは特記のない限り byte address、容量の KiB/MiB は 2 進接頭辞である。

採用情報の優先順位

構成指定、現行 RTL と C/C++ コード、Markdown 説明、図中の概念表記の順で照合した。画像と文書に残る旧設定を、現行回路の定数として転記していない。各頁の根拠番号と最終頁の対応表から主要ソースを追跡できる。

1　システム概要と構成境界

PSC-ONE は Tang Nano 20K を中心とする FPGA SoC である。CPU 単体の命令処理と、SoC のキャッシュ・外部メモリ・周辺回路を分けて理解する。FPGA トップは `PSC_ONE_Chip`、内部 SoC は `PSC_ONE_SOC` である。

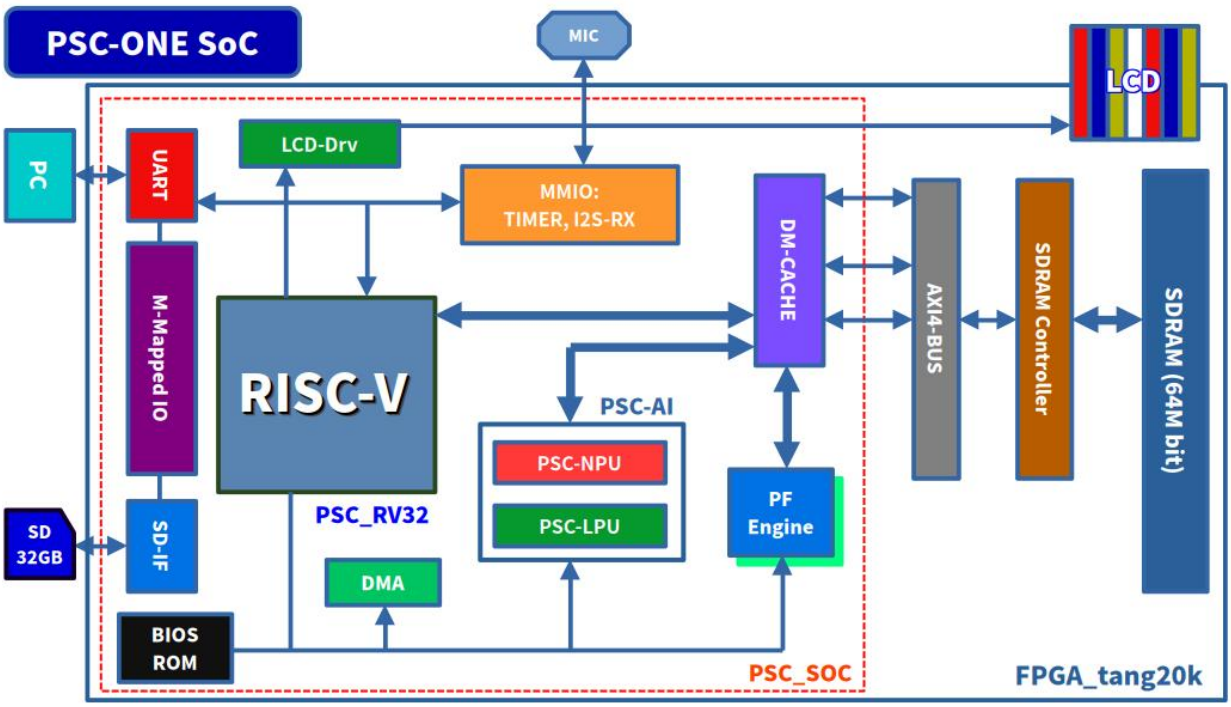


図 2: SoC 全体の構成概念。SD 容量 32 GB 等は図示例であり、必須容量ではない。

主な階層とデータ経路

階層	機能と接続
CPU core	命令取得、演算、整数レジスタ、CSR、trap、命令／データ MMU。
CPU wrapper	I-cache、D-cache、NPU 制御、256-bit ラインと 32-bit AXI の接続。
SoC	UART、timer、LCD、SD、I2S、I2C、PIO、LED、PFE、DMA、boot 制御。
FPGA top	PLL、SDRAM 制御、SDRAM 端子、SoC と外部基板信号の接続。
PSC-OS	初期化、ページ管理、ECALL、ドライバ、シェル、組込みライブラリ。

アクセラレータの接続

SynapEngine は CPU のカスタム CSR で起動する。行列アクセスは CPU と共通の D-cache コントローラに入る。独立した転送用 DMA もあるが、NPU のアクセス経路と同一視しない。PFE は QUBO 演算の実験回路であり、公開 OS API の有無は別に確認する。図中の PSC-LPU は選択可能な派生回路で、標準 legacy NPU との同時動作を意味しない。

根拠：S01、S02、S03、S10。資料番号は最終頁を参照。

2　CPU アーキテクチャ

安定版 CPU v1 は valid/ready、forwarding、stall 制御によって命令取得、デコード、演算、メモリアクセス、commit を重ねるパイプライン CPU である。命令取得と実行側に独立した MMU を置き、命令 PC は仮想アドレスとして保持する。

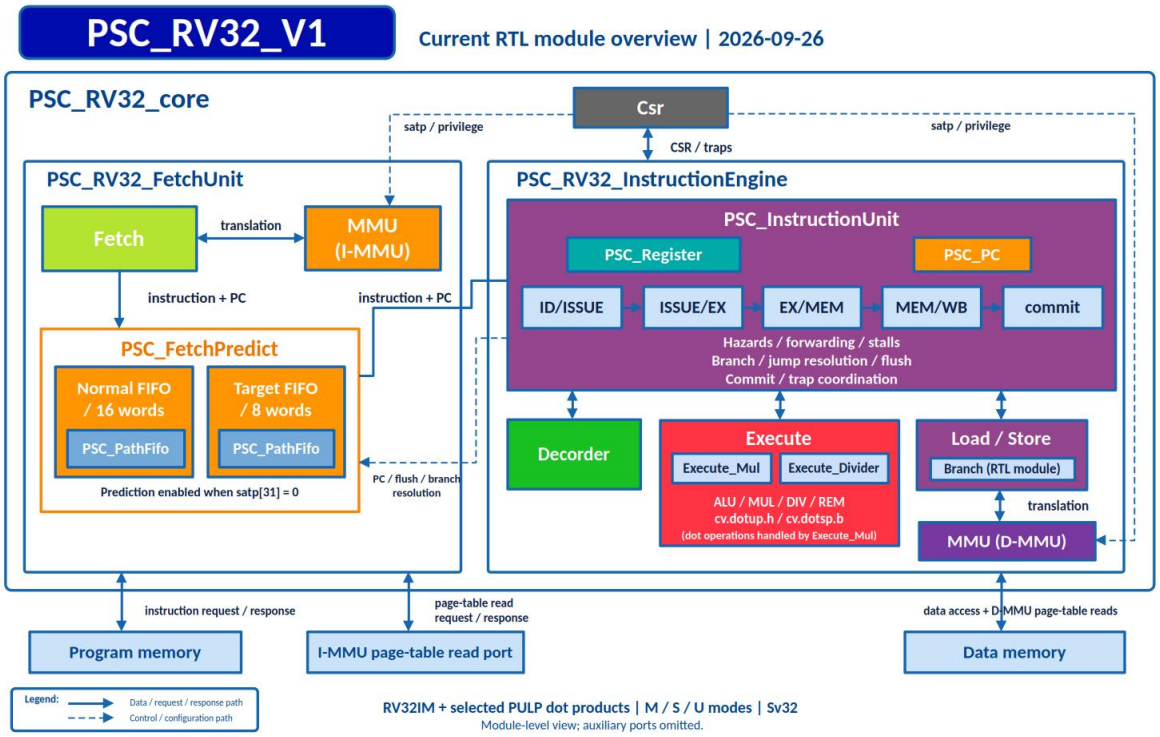


図 3: 現行 CPU v1 のモジュール構成図。通常 FIFO 16 words、予測先 FIFO 8 words。

CPU	位置付け	実行方式	主な相違
legacy	初期実装・比較用	状態制御、選択的な重ね合わせ	回帰の基準となる旧 CPU
v1	安定版・本書の中心	パイプライン	分岐予測、依存解決、選択的 PULP
v2	実験版	小規模 out-of-order	ROB=2、IQ=2、物理レジスタ 34

命令取得と分岐

v1 の `FIFO_DEPTH` 既定値は 16。予測回路は通常経路と予測先経路を保持する。現行接続では `satp[31]=0` のとき分岐予測を許可し、ページ変換有効時は抑止する。分岐結果の確定や trap に伴って経路の破棄・PC 切替を行う。過去 Markdown の「図は 32 words」という注記と異なり、今回参照した JPG 自体は 16 words へ更新されている。

v2 の解釈

v2 はレジスタ名の付替えと順序内 retirement を備える。準備の整った独立命令は先行できるが、2 エントリの ROB は 1 クロックで 2 命令完了する保証ではない。メモリと CSR の副作用は ROB 先頭で制御する。

根拠：S04、S05。資料番号は最終頁を参照。

3　命令セット、CSR、例外

分類	CPU v1 の実装範囲
整数演算	RV32I の算術、論理、比較、シフト、分岐、ジャンプ、load/store。
乗除算	整数乗算、除算、剰余。専用乗算・除算モジュールを使用。
制御	CSR アクセス、FENCE、FENCE.I、ECALL、特権復帰、SFENCE.VMA。
特権	M/S/U の mode、trap、例外・割込み処理。完全な保護準拠とは別。
PULP 部分拡張	`cv.dotup.h`、`cv.dotsp.b`。全 CORE-V 命令への対応ではない。
アクセス整列	自然整列を前提とする。misaligned data テストは既存 ISA スイートから除外。

2 命令の SIMD 仕様

`cv.dotup.h` は 2 組の unsigned 16-bit 積を加算し、下位 32 bit を rd へ書く。`cv.dotsp.b` は 4 組の signed 8-bit 積の和を rd へ書く。いずれも旧 rd を足す積和命令ではない。x0 への書込み、依存関係、例外処理は CPU の通常制御に従う。

$$\text{DOTUP.H}: \quad rd = (u_{16}(a_0)u_{16}(b_0) + u_{16}(a_1)u_{16}(b_1)) \bmod 2^{32}$$

$$\text{DOTSP.B}: \quad rd = \sum_{i=0}^{3} s_8(a_i)s_8(b_i)$$

命令	opcode / funct3 / funct7	MATCH	MASK
`cv.dotup.h`	0x7b / 0 / 0x40	`0x8000007b`	`0xfe00707f`
`cv.dotsp.b`	0x7b / 1 / 0x48	`0x9000107b`	`0xfe00707f`

```
.insn r 0x7b, 0, 0x40, rd, rs1, rs2  # cv.dotup.h
.insn r 0x7b, 1, 0x48, rd, rs1, rs2  # cv.dotsp.b
```

`cv.sdotsp.b` は未実装で Illegal Instruction となる。コンパイラやライブラリの「PULP 対応」指定だけで任意の拡張命令を使用しない。

CSR と trap の責務

M/S-mode CSR は状態、例外 PC、cause、trap value、trap 入口、SATP 等を保持する。MMU は変換結果と fault を通知し、CPU が trap CSR と制御を更新する。命令 page fault は cause 12、load は 13、store は 15 である。NPU・DMA・D-cache のカスタム CSR は後述の専用制御を担う。

根拠：S04、S05、S06。資料番号は最終頁を参照。

4　対応 PULP 命令の詳細仕様と使用例

対象は CPU v1 の `ENABLE_PULP=1` で有効となる 2 命令である。CPU レジスタ上のドット積であり、NPU の CSR 起動やメモリ転送を伴わない。

共通の命令形式

bit	31:25	24:20	19:15	14:12	11:7	6:0
field	funct7	rs2	rs1	funct3	rd	opcode
`dotup.h`	0x40	入力 2	入力 1	0	出力	0x7b
`dotsp.b`	0x48	入力 2	入力 1	1	出力	0x7b

命令語は `MATCH | (rs2<<20) | (rs1<<15) | (rd<<7)` で生成する。rs1/rs2/rd は 5-bit のレジスタ番号。MATCH/MASK は前頁の値を用いる。

cv.dotup.h：unsigned halfword 2-lane

rs1/rs2 の bits[15:0] を lane 0、bits[31:16] を lane 1 とし、各 lane を 0〜65535 と解釈する。各積は 32 bit、2 積の和は 33 bit で、rd へは下位 32 bit を格納する。最大和は 8,589,672,450（`0x1FFFC0002`）。上位 carry は捨てる。

rs1	rs2	rd	確認点
`0x00010002`	`0x00030004`	`0x0000000B`	$2 \times 4 + 1 \times 3 = 11$
`0x80008000`	`0x00020003`	`0x00028000`	各 0x8000 は unsigned 32768
`0xFFFFFFFF`	`0xFFFFFFFF`	`0xFFFC0002`	33-bit 和の下位 32 bit

cv.dotsp.b：signed byte 4-lane

各 bits$[8i+7:8i]$（$i = 0, 1, 2, 3$）を独立した 2 の補数 INT8、− 128〜127 と解釈する。4 積を signed 加算して 32 bit へ符号拡張する。積は 16 bit、2 積の和は 17 bit、4 積の和は 18 bit で、結果範囲は − 65024〜65536。byte 間の交差項は加えない。

rs1	rs2	rd	確認点
`0x04030201`	`0x01010101`	`0x0000000A`	$1 + 2 + 3 + 4 = 10$
`0x80808080`	`0x7F7F7F7F`	`0xFFFF0200`	$4 \times (-128) \times 127 = -65024$
`0x80808080`	`0x80808080`	`0x00010000`	$4 \times (-128)^2 = 65536$

レジスタと副作用

旧 rd の加算、飽和、丸め、演算 flag 更新はない。rs1/rs2=x0 は 0、rd=x0 は結果を破棄する。rd=rs1、rd=rs2、3 レジスタ同一でも書戻し前の入力で計算する。通常の依存解決・commit を通り、命令固有の CSR 変更やメモリアクセスはない。機能の有無は build 構成で管理し、MISA の標準 bit から検出しない。ニーモニック非対応の assembler では前頁の `.insn` 形式を使用する。無効構成や未対応の `cv.sdotsp.b` 等は Illegal Instruction となる。

根拠：S05（Decorder.sv、Execute_Mul.sv）、S06。資料番号は最終頁を参照。

5　Sv32 MMU とメモリ保護の範囲

変換条件とアドレス分割

`satp[31]=0` または M-mode では変換をバイパスする。MODE=1 かつ U/S-mode では 2 段ページテーブルを参照する。ルートは `satp[21:0] << 12`、各テーブルは 1,024 個の 32-bit PTE で 4 KiB。

仮想アドレス	用途	幅
VA[31:22]	上位テーブル index：VPN[1]	10 bit
VA[21:12]	下位テーブル index：VPN[0]	10 bit
VA[11:0]	ページ内 offset	12 bit

ページテーブルウォーク

上位 PTE をルート＋ VPN[1]×4 から読む。leaf なら 4 MiB ページ、非 leaf なら下位 PTE へ進む。下位 leaf は 4 KiB ページを表す。V=0、または W=1 かつ R=0 を不正とし、要求する R/W/X を検査する。4 MiB ページでは PPN[0]=0 の整列条件も検査する。内部に 34-bit の信号があっても、外部物理アドレス出力は下位 32 bit である。

変換キャッシュと同期

上位・下位の PTE を各 1 エントリ保持する。上位のキーはルート PPN と VPN[1]、下位は上位 hit と VPN[0] で判定する。多エントリの ASID 付き TLB ではない。reset と `SFENCE.VMA` で無効化する。v1/v2 は要求をラッチし idle で処理する。PTE を書き換えた後は変換キャッシュの同期が必要で、コード転送後の I/D-cache 同期とは別である。

```
# PSC-OS ELF address-space switch (ordering summary)
fence rw, rw
sfence.vma
csrw satp, <new_root>
sfence.vma
fence.i
```

実装上の制限

- U bit による U/S アクセス制限、A/D 検査と自動更新を実装しない。
- SUM、MXR、MPRV に基づく実効権限制御は MMU 入力にない。
- ASID tagging、global ページ最適化、VA/ASID 別 flush を行わない。
- PTE 読出しの bus-error 入力・タイムアウトを持たず、応答待ちは固定時間ではない。

したがってユーザー／カーネルの mode 区分や ELF 検査があっても、信頼できないコードの完全な隔離境界とはならない。fault 時は `mmu_done` だけでなく `page_fault` も確認する。

根拠：S05、S07。資料番号は最終頁を参照。

6　キャッシュ、SDRAM、DMA

現行 CPU v1 のキャッシュ容量

ラインは 32 bytes（256 bit）、direct-mapped。容量は define 名の「WORD」表記でなく、`TAGLSB` と offset 5 bit から算出する。現行 wrapper の有効 define を基準とする。

経路	TAGLSB	ライン数	データ容量	動作
I-cache	12	$2^{12-5} = 128$	4 KiB	命令取得、8 words burst
D-cache	10	$2^{10-5} = 32$	1 KiB	write-back / write-allocate
define 未選択時	14	512	16 KiB	汎用既定値。現行構成と区別

D-cache の store miss は必要なら dirty victim を書き戻し、新しいラインを読んでから byte/halfword/word 書込みを反映する。旧 README の write-no-allocate 記述より現行 RTL を優先する。2-line のレジスタ読出しキャッシュも備える。MMIO は通常キャッシュ格納を迂回する。

一貫性とソフトウェア制御

CPU と NPU は同じ D-cache tag/data 経路を使用するため、CPU の dirty 入力を NPU が同じ経路から参照できる。全マスターを自動 snoop する一般的な coherent fabric ではない。独立 DMA や命令公開では専用同期を行う。

CSR	名称	用途
`0x7F0`	D-cache 制御	bit 0：clear、bit 1：write-back 要求。
`0x7E0`	DMA 制御	bit 0：転送開始。
`0x7E4`	DMA 長	転送 word 数。byte 数をそのまま設定しない。
`0x7E8`	DMA SRC	読出し元アドレス。
`0x7EC`	DMA DST	書込み先アドレス。

SDRAM とバス

対象は 64 Mbit（8 MiB）の SDR SDRAM。トップ端子は DQ=32 bit、address=11 bit、bank=2 bit、DQM=4 bit であり、旧 TeX の 16-bit DQ 記載を置き換える。SoC は命令、データ、DMA、boot の 32-bit AXI 経路を持つ。キャッシュ側 256-bit ラインと AXI 側の幅は異なる。SDRAM の初期化、refresh、row/column 操作は専用 controller が処理する。

動作クロックの扱い

トップの `CLK_FREQ=80` は MHz 単位の設定値で、最大保証周波数ではない。内部名 `clock_100MHz` も実周波数の証拠にはならない。PLL、タイマー分周、SDRAM timing と配置配線制約を揃えてビルドする。

根拠：S02、S03、S05、S08、S09。資料番号は最終頁を参照。

7　メモリ配置と MMIO アドレス一覧

PSC-OS 既定配置

アドレス／範囲	用途
0x00200000	FAT32 boot での KERNEL.MEM ロード先。
0x00400000	USER.MEM ロード先／通常ユーザー仮想領域の先頭。
0x00400000--0x004FFFFF	PSC 構成のユーザー VA 枠（1 MiB）。全域常時マップとは限らない。
0x00030000	現行 Synap C ドライバが使用する結果作業領域。

ロード先物理アドレスと実行中の仮想アドレスを混同しない。カーネルはユーザーコードを実行用ページへコピーする。実際の配置はリンカとビルド定義に従う。

SoC トップの既定値

アドレス	レジスタ／ブロック	説明
0x10000000	UART TX	送信 byte
0x10000004	UART RX	受信 byte
0x10000008 / 0C	UART ST / CT	状態／制御（0C は末尾 offset）
0x10001000	PIO	read：SW 等の入力、write：出力ラッチ
0x10002000	TIMER WRITE	reload と start/stop 等の制御
0x10002004 / 08	TIMER COUNT / ST	counter ／状態
0x10003000 / 04	LCD DATA / ST	描画データ／状態
0x10004000	LED	6-bit LED 出力
0x10006000	SD DATA	FIFO pop/push、下位 byte
0x10006004 / 08	SD SECTOR / CTRL	LBA ／制御・状態
0x10007000 / 04	I2S RX / ST	サンプル／ FIFO 状態・flush
0x10008000 / 04	PFE DATA / CTRL	実験用 QUBO 回路
0x10009000	I2C INPUT	4 個の INT8 入力
0x10009004 / 08	I2C STATUS / OUTPUT	フラグ／ INT8 結果
0x1000900C / 10	I2C CONTROL / SEQ	IRQ 許可／ 8-bit sequence

アクセス規約

表は SoC で上書きされた値であり、単体モジュールの parameter 既定値とは異なる場合がある。通常は 32-bit MMIO アクセスで所定の下位 bit を使用する。DATA の読出しには FIFO 消費等の副作用がある。未定義アドレスの正常応答は保証しない。旧 0x10005000 の SA 定義がヘッダに残るが、現行 SoC の SA MMIO parameter は 0 であり、NPU 制御は 0x7C0 系 CSR を使用する。

根拠：S03、S09、S12、S14。資料番号は最終頁を参照。

8　UART、タイマー、GPIO、I2C

UART コンソール

実機既定は 115200 bit/s、8 data bits、no parity、1 stop bit（8N1）。`FST_UART_MODE` では高速化用 baud 値に切り替わるため、シミュレーション値を実機設定へ流用しない。TX は送信要求と 1-byte バッファ、RX は受信値と状態を保持する。ST[0]=tx busy、[1]=rx available、[2]=IRQ、[3]=overrun、[4]=tx buffer valid。RX 読出しは受信状態を消費する。UART の IRQ 出力は現行 SoC で未接続である。

16-bit タイマー

`FRAC=1` の SoC 構成は 1 tick=1 µs を意図する。16-bit counter を count down し、単発または autoreload で動作する。周波数設定が実クロックと一致することが前提。

WRITE bit	内容
[15:0]	reload 値。start 時の 0 は 1 へ置換。
16 / 17 / 18	start / autoreload / IRQ enable
19 / 20	stop / clear IRQ（write 1 で clear）

長時間待機は OS ラッパーを使い、1 回の 16-bit 計数範囲と区別する。同じタイマーを待機、計測、割込みに併用する場合は所有権を調整する。LED は 6 本、PIO 下位 2 bit はボードの 2 個のスイッチ入力として利用される。

I2C slave による入出力

7-bit slave address は `0x42`。外部 master はレジスタ `0x00` に続けて signed INT8 を 4 個送信し、CPU が処理した INT8 結果を `0x08` から読む。状態は `0x04` で確認する。バス上のレジスタ番号と CPU MMIO アドレスは別体系である。

状態／制御	意味
STATUS bit 0	INPUT_READY：4-byte 入力の完了。
STATUS bit 1	OUTPUT_READY：CPU の結果書込み完了。
STATUS bit 2	OVERRUN：前入力が未処理の状態で更新。フラグは W1C。
SEQ	完全な 4-byte 入力ごとに増加、8-bit wrap。
IRQ enable	周辺 IRQ 生成を許可するが、現行 CPU へは未接続。polling を使用。

入力は byte 受信中にも更新され、FIFO や一括 snapshot ではない。master は結果を読み終えてから次の入力を送る。SDA は low/Hi-Z の open-drain 出力として実装される。`i2c_init()`、`i2c_wait_input()`、`i2c_read_input()`、`i2c_write_output()` がカーネル用 API で、ECALL ／ Python 公開はない。

根拠：S03、S09、S15。資料番号は最終頁を参照。

9　SD カード、FAT32、ブート

SDHC の SPI 転送

SD controller は CMD17 による単一 512-byte 読出しと CMD24 による単一 512-byte 書込みを行う。SPI engine、512-byte FIFO、状態機械から構成される。SDHC の LBA addressing が前提である。初期化は 80 clocks、CMD0、CMD8、CMD55/ACMD41、CMD58 を経て READY へ進む。

CTRL への write	bit 0：init、1：read、2：FIFO flush、3：soft reset、4：write、7：local error clear。1 操作ずつ発行する。
CTRL から read	bit 1：busy、2：ready、3：empty、4：full、5：error。上位 byte に受信 CRC を保持。

`busy=(state!=READY)||(fifo_count!=0)` であり、未読データが残っている間も busy になる。読出し前に busy=0 だけを待たず、empty と error を確認しながら FIFO を排出する。RTL は受信 CRC を保持し、検証はソフトウェア側が担う。書込みは既定で LBA>`0x1000` に制限される。bit 7 だけで全 FSM エラーの解除を保証しない。

起動経路と生成物

1. FPGA 構成後、SDRAM と boot 制御が初期化される。
2. M-mode BootROM / MBIOS が特権状態を設定し、S-mode bootloader へ移る。
3. FAT32 root の `KERNEL.MEM` を `0x00200000`、`USER.MEM` を `0x00400000` へロードする。
4. kernel がページテーブル、idle、shell を準備し、ユーザーコードの cache 同期を行う。
5. U-mode shell が UART へ `PSC_OS>`を表示する。

raw-sector 版は別の配置方式である。FAT32 上の MEM ファイルと raw IMG を混在させず、FPGA ROM 初期値、bootloader、SD 内イメージを揃える。

FAT32 のファイル規約

`fat32_ls()` は LFN の表示と UTF-8 出力に対応する。一方、モデル、ELF、stream API の open は root 内の大文字 8.3 名が基本である。`MODEL.TFL`、`HELLO.ELF` 等を用いる。一覧に長いファイル名が表示されても、その名前や任意パスで open できるとは限らない。`fat32_stream_read()` の短い正常読出し／0 bytes は EOF で、負値はエラーを表す。

障害切り分け

無出力なら UART 設定、reset と PLL、bitstream を確認する。boot 途中停止なら SD 初期化・FAT32・ファイル名・ロード先・ROM 初期値を照合する。raw 書込み試験はファイルシステムの通常操作ではなく、試験用カード・領域に限定する。

根拠：S09、S13、S14、S15。資料番号は最終頁を参照。

10　表示、JPEG、I2S オーディオ



図 4: 既存の実機ブート表示写真。過去の表示記録であり、本書作成時の実機試験結果ではない。

LCD 表示経路

SPI LCD controller と ILI9488 向けドライバを使用する。JPEG 表示は 480×320 の landscape、RGB888 で転送する。`JPEG_LCD_MADCTL=0xE8` が現行設定である。文字表示系には別の 320×480 定義があり、画面方向を混同しない。ユーザー描画は `SYS_LCD_RGB888_BEGIN` と `SYS_LCD_RGB888_RECT` を用い、1 矩形を最大 16×16 pixels としてカーネルへコピーする。

JPEG 仕様

`jpeg TEST.JPG` で原寸表示する。Baseline JPEG、幅 1〜480、高さ 1〜320 を対象とし、TJpgDec による逐次デコードで全画面 framebuffer を必須としない。対応サブサンプリングや成分 ID には制約があり、Progressive、CMYK/YCCK、12-bit、Lossless、多重 scan 等は対象外。サイズ超過や未対応形式はエラーとして扱う。JPEG 全体の画素内容を保証する checksum はない。

I2S 入力

SoC は 64-entry、24-bit サンプル保持用 FIFO を構成する。I2S clock/word select を出力し、マイクの serial data を取り込む。モノラル 16 kHz を意図した整数分周で、`I2S_SCK_DIV=CLK_FREQ*1000000/(2*1024000)`。丸めがあるため実際のサンプル周波数は入力 clock と分周比に依存する。`I2S_LR=0`。ST には empty/full/count があり、write bit 0 で FIFO を flush する。音声認識デモの存在は、任意モデル・任意マイクでの認識精度の保証ではない。

根拠：S03、S09、S16。資料番号は最終頁を参照。

11　標準 NPU：SynapEngine legacy

PSC-ONE の標準 NPU は legacy である。`hardware/rtl/soc/npu/`の controller wrapper は既定で `ENABLE_NPU=1`、`ENABLE_LPU=0`、`MUL_NUM=4`。CPU 側は `PSC_NPU_Controller` を接続する。

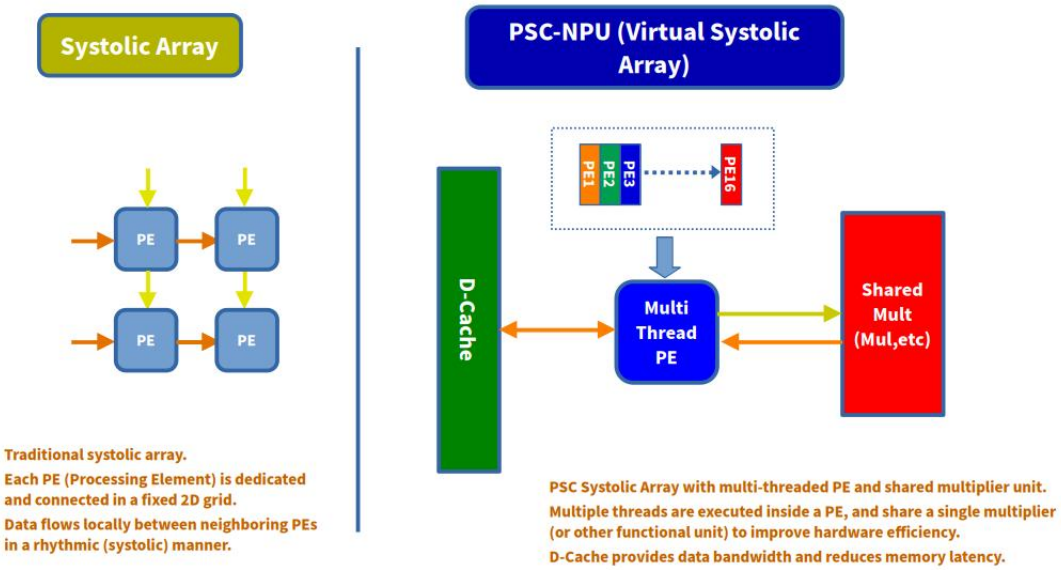


図 5: 論理 PE と共有演算器を分離する概念図。「single multiplier」は概念表現で、既定は 4 基。

論理配列と演算

論理 4×4 の 16 PE context を維持し、A を横、B を縦に流す Output-Stationary 方式。部分和は各 PE で保持し、完成した INT32 結果を共有メモリへ戻す。物理乗算器数は論理 PE 数と独立で、legacy では parameter で変更できる。INT8 入力は signed/unsigned を選べる。累積は 32-bit で、飽和 INT8 化や量子化後処理は標準 NPU の出力機能ではない。

$$C_{M\times N} = A_{M\times K} B_{K\times N}, \qquad C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}$$

項目	legacy controller の範囲
M（A/C 行数）	1〜255。0 は従来互換の M=N。
K、N	4〜252 の 4 の倍数。タイルは論理 4×4。
メモリ配置	A/B は row-major の連続 bytes、C は row-major の連続 32-bit 値。
端数行	無効 A 行を 0 として扱い、有効 C 行だけを書込む。余分な行の padding は不要。
演算形態	現行は Output-Stationary。Weight-Stationary や FP16/BF16 は本構成の機能ではない。

M は start 時に捕捉する。K/N、ベースアドレス、signed 設定は完了まで安定させる。A/B 読出しは engine で `RADDR_MASK=0x00FFFFFF` を設定するため、上位アドレスの扱いに注意し、既存の物理バッファ API を使用する。

根拠：S05、S10、S12。資料番号は最終頁を参照。

12　NPU 制御 CSR とソフトウェア契約

CSR	名前	現行 CPU v1 からの接続
0x7C0	SA CTRL	[0]start、[1]state reset、[2]clear、[3]signed、[4]OS mode、[11:8]OS instruction。
0x7C4	SA SIZE	[7:0]=K、[15:8]=N、[23:16]=M。
0x7C8	SA STATUS	[0]=done、[1]=busy。状態を読出す。
0x7D0	SA ADDR A	入力 A ベース。
0x7D4	SA ADDR B	入力 B ベース。
0x7D8	SA ADDR C	出力 C ベース。

Markdown の旧記載「CSR から M を駆動しない」と異なり、現行 CPU v1 は `SA_SIZE[23:16]` を `matrix_size_m` へ接続している。標準 C ドライバも M/K/N を格納する。この差異は現行コードを採用する。

推奨呼出し経路

U-mode の組込みアプリは `call_sa_matmul_int8_rect()` を使う。ECALL が A/B を kernel バッファへコピーし、カーネルの `sa_run_rect_checked()` が CSR を操作する。結果は 0x00030000 から読出してユーザー C へ返す。ユーザー VA をそのまま NPU の物理ポインタとして渡さない。

1. 寸法、ポインタ、C の 4-byte 整列、転送範囲を検査し、入力をコピーする。
2. A/B/C アドレス、SIZE、signed と OS mode を設定し、`fence rw,rw` を実行する。
3. state reset、clear をそれぞれ assert/deassert してから start を設定する。
4. STATUS done を polling する。現行ドライバの上限は 10,000,000 反復で、時間単位ではない。
5. start を落とし、fence 後に C をコピーする。失敗した結果を使用しない。

ハードウェア上限と API 上限

層	利用範囲
legacy RTL	M ≤ 255、K/N ≤ 252（4 の倍数）。
現行 C ／ ECALL	`SA_MAT_MAX=64`。M=1〜64、K/N=4〜64 の 4 の倍数。
MicroPython 行列 API	既定 `PSC_SA_MAT_MAX=16`。正方形 4/8/12/16。

長方形 ECALL の形式

`SYS_SA_RUN=10`、a0=A、a1=B、a2=C。長方形では a5 に `PSC_SA_RECT_FLAG=0x40000000` を付け、a4=$M \ll 16 \mid N \ll 8 \mid K$ とする。flag なしは従来の正方形寸法。a5 bit 0 は signed、bit 31 は a6 への profile 出力を有効化する。専用バッファと実行状態を共有するため並列・再入呼出しは避ける。

根拠：S05、S10、S12、S15。資料番号は最終頁を参照。

13 派生アクセラレータと実験機能

構成	演算回路	用途・特徴	本書の位置付け
NPU legacy	共有乗算器、既定 4 基	論理 4×4、INT8 行列積	標準
NPU v1	固定 4-lane 乗算	16 accumulator へ順次 writeback	比較・派生
NPU v2	shift/add 系実験	独自 encoded weight 演算	実験版
PSC-LPU	共有加算 4 lanes	ternary 重み、INT8 入力	選択式の実験版
PFE	QUBO energy engine	既定 8 変数、MMIO 制御	開発中

NPU v1 と標準 legacy の違い

v1 は `MUL_NUM` を持たず、lane n が PE ID $n, n+4, n+8, n+12$ を担当する。operand snapshot、乗算、registered writeback、accumulation を分離する。1 batch の start 受理を E0 とすると E2 で operand 捕捉、E3〜E6 で writeback、E4〜E7 で累積、E8 で done となる。この値は MAC batch の内部タイミングであり、行列全体の転送を含む処理時間ではない。busy 中の部分和を確定結果として読まない。

LPU の構成と重み形式

legacy ディレクトリ内の wrapper で `ENABLE_NPU=0`、`ENABLE_LPU=1` を指定する。両方 1 は構成エラー、両方 0 は出力 0。標準 NPU と別の状態機械を選択する方式である。A は signed INT8、B は 4×4 tile を 1 つの 32-bit word に詰めた 2-bit ternary で、最初の要素が bits[1:0]。00=0、01=+1、10=− 1、11 は予約（RTL では 0 扱い）である。

$$\mathrm{Btile}(k_t, n_t) = \mathrm{BASE_B} + 4\{k_t(N/4) + n_t\}$$

16 個の signed INT32 accumulator と 4 本の加算 lane を持つ。既定出力は `OUT_SHIFT` による算術右 shift の後、− 128〜127 へ飽和して INT8 にする。`LPU_OUTPUT_INT32=1` では生の 32-bit accumulator を書き出す。NPU の通常 INT8 重みや INT32 出力バッファと互換だと仮定しない。丸め、bias、ReLU、zero-point 補正、TFLite requantization はこの回路に含まれない。

PFE

`PSC_PFE.v` は係数・入力の load、start、clear と energy/status 読出しを持つ。DATA=`0x10008000`、CTRL=`0x10008004`。OS syscall dispatcher と MicroPython `psc` には PFE API がなく、ハードウェアの存在だけで通常アプリから利用できる機能とは扱わない。

ビルド上の注意

CPU 版と NPU 版は独立して指定する。複数版の NPU は公開 module 名を共用するため、同一コンパイルに混ぜない。標準構成を再現するコマンドでは `NPU_VERSION=legacy` を明記する。

根拠：S10、S11、S18。資料番号は最終頁を参照。

14　PSC-OS の構造とタスク実行

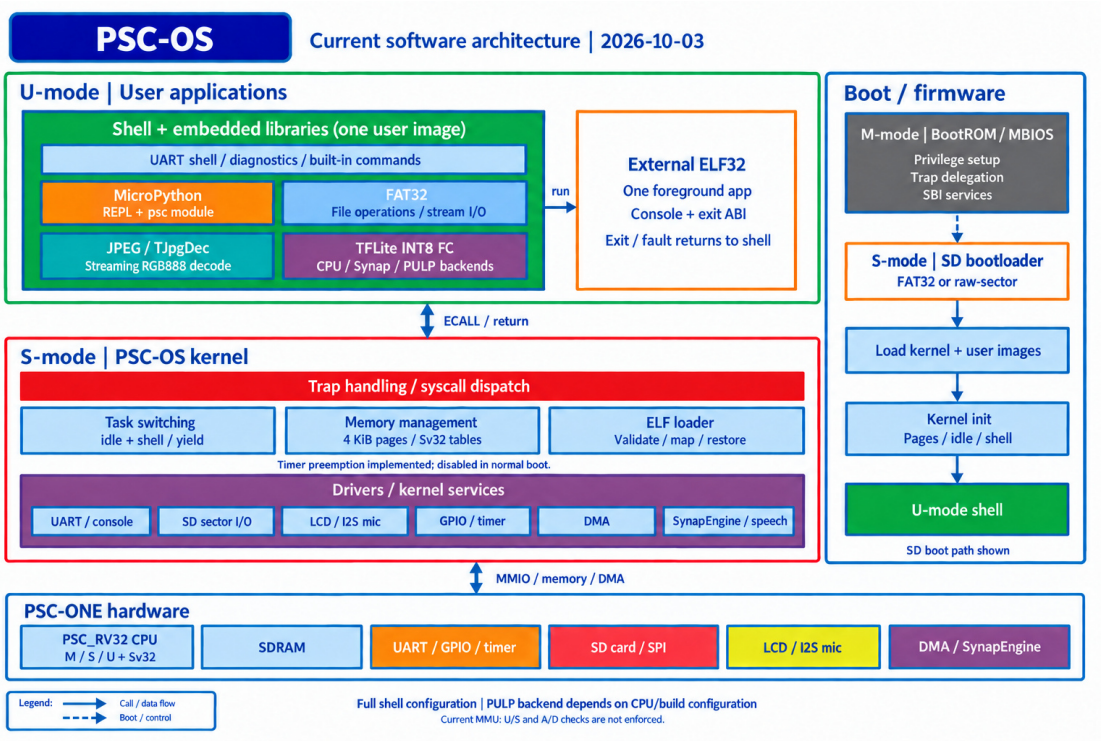


図 6: PSC-OS の実装階層。mode 分割は現行 MMU による完全な隔離保証を意味しない。

PSC-OS は独自の小規模 OS であり、Linux や既存 RTOS を基盤としない。M-mode の低レベル処理、S-mode カーネル、U-mode シェルを組み合わせる。FAT32、MicroPython、JPEG、TFLite は full shell のユーザーイメージにリンクされ、SD 等のデバイス I/O は ECALL 経由でカーネルを呼ぶ。

タスクと割込み

通常起動で idle と shell を生成する。`yield()` は実行可能タスクを選び、ページテーブルと context を切り替える。M-mode timer trap によるプリエンプション経路と 1 ms tick を備えるが、現行 `MULTITASK_MODE=0` では追加 kernel task と `preemption_start()` が通常起動で無効である。

メモリ管理

`alloc_pages()` はゼロ初期化付きの前進型アロケータであり、一般的な free、swap、demand paging を提供しない。通常シェル stack は 128 KiB。U-mode trap はユーザー SP をカーネル stack として使わず、プロセス用 8 KiB stack へ切り替え、整数レジスタと SEPC/SSTATUS/cause/value を保存する。

提供範囲

コンソール、組込みアプリ、foreground ELF、推論・描画デモを中心とした環境である。POSIX ABI、fork、汎用 execve、動的リンク、優先度・期限保証を前提としない。シェルの診断 API には raw pointer を扱うものもあり、全 API で同一のポインタ検査をするわけではない。

根拠：S14、S15。資料番号は最終頁を参照。

15　ECALL ABI と外部 ELF

独自 ECALL 規約

番号は **a3**、基本引数は a0〜a2、追加引数に a4〜a6 を使う。戻り値は通常 a0、ELF 起動は a1 にも exit code を返す。Linux の a7 番号規約と異なる。

番号	サービス	主な機能
1 / 2 / 3	PUTCHAR / GETCHAR / TIMEOUT	文字出力、入力、タイムアウト入力
10	SA RUN	NPU 行列演算、任意 profile
20 / 21	I2S MIC READ / WRITE	音声処理用サービス
30〜33	SD 操作	セクタ診断・試験書込み・buffer I/O
40 / 50 / 60	DUMP / SW / SPEECH	メモリ診断、SW 読出し、音声認識
70〜79、87、88	TIMER	start/stop、count、待機、計測
80〜86	LED	値設定、on/off/toggle、全体操作
92 / 93	LCD RGB888	描画開始／矩形転送
100 / 101 / 102	EXIT / PRINT INT / ELF RUN	終了、整数表示、ELF 起動

`READFILE=51`、`WRITEFILE=52`、`LCD_INIT=90`、`LCD_FILL_RGB=91` は番号定義があっても現行 dispatch がない。通常シェルの未知の番号は PANIC になり得る。既存ラッパーから呼び出す。

外部 ELF のローダ契約

`run HELLO.ELF` は FAT32 から最大 32 KiB の ELF32 RISC-V ET_EXEC を読み、program header、PT_LOAD、範囲、整列、権限、重複、entry を検査する。最大 8 program headers、load 用 16 pages（64 KiB）。BSS とページ余白を 0 にする。

仮想領域	ELF 実行中の用途
`0x00400000--0x004FAFFF`	必要な PT_LOAD ページのみを map。
`0x004FB000--0x004FBFFF`	未マップの stack guard。
`0x004FC000--0x004FFFFF`	16 KiB stack、U/R/W、X なし。

ELF は同時に 1 つの foreground 実行。entry は実行可能な file-backed 領域内とする。元の shell trap frame と root を保存し、EXIT または処理可能な fault で復帰する。別 scheduler process の生成ではない。PIE、relocation、共有ライブラリ、TLS、引数・環境変数、kill ／実行時間 timeout は未対応。

ELF に許可するサービス

番号 1、2、3、100、101 のみ。その他は a0=−1 を返す。組込みシェルが使う SD、NPU、LCD 等を外部 ELF へそのまま公開していない。この制限と guard page は MMU の U/S 検査不足を解消するものではない。

根拠：S14、S15、S17。資料番号は最終頁を参照。

16 MicroPython と INT8 推論ランタイム

MicroPython PSC ポート

シェルの `micropython` から REPL へ入り `import psc` を実行する。独立した汎用 Python OS ではなく、シェル内に組み込まれたランタイムである。

```
import psc
psc.led_on(0)
psc.wait_ms(100)
psc.led_off(0)
A = [[1 if i == j else 0 for j in range(4)] for i in range(4)]
B = [[i * 4 + j for j in range(4)] for i in range(4)]
print(psc.sa_run(A, B, True))
```

`psc.run(name)` は SD 上の最大 4 KiB の Python スクリプトを実行する。行列 API は正方形 4/8/12/16、signed 時 −128〜127、unsigned 時 0〜255。入力不正は ValueError、デバイス失敗は OSError を用いる。

TFLite 対応範囲

TFLite 形式の検証と限定 INT8 Fully Connected（FC）実行を提供する。汎用 TFLM interpreter 全体や任意ニューラルネットの実行を提供するものではない。モデルファイル上限 8 KiB、tensor arena 4 KiB、単一 resident model、同期・非再入が基本契約。固定 shape、INT8 activations/weights、任意 INT32 bias、per-tensor 量子化等を検査する。FC の NONE/RELU を扱い、未対応 operator を黙って読み飛ばさない。

現行 C backend	意味
0：CPU	参照 CPU 演算。load/prepare/reset 後の既定。
1：Synap	長方形 SA 転送を使う NPU 演算。失敗時に暗黙 CPU fallback しない。
2：PULP	対応 CPU ／ build で SIMD 演算。利用不可時は CPU へ fallback。
3：AUTO	M=K=N=16 または 32 で NPU を選ぶ。それ以外は PULP ／ CPU。

現行 C の tile 指定は 4〜64 の 4 の倍数。Python の公開 backend は 0/1 のみ、tile 指定は 4/8/12/16 に限定される。古い Phase 4 文書の正方形専用転送より、現行 `tflite_synap.cc` は有効入力行 M だけを使う長方形・batch 転送へ進んでいる。

実行とデータ寿命

`load/prepare` → `get_input` → `invoke` → `get_output` の順に使う。prepare に渡すモデルは整列済み・不変の領域を保持する。reset/load/prepare/inspect 後は旧入出力ポインタを使用しない。出力は invoke 成功後だけ有効。NPU は raw dot を担い、zero-point 補正、bias、requantization、clamp 等を CPU 側で行う。`tflite_info` は構造検査であり推論実行ではない。

根拠：S12、S15、S19。資料番号は最終頁を参照。

17　ボード端子、実装条件、ビルド

主要端子（Tang Nano 20K 用 CST）

次表の番号は FPGA package pin であり、基板 connector 番号ではない。CST は GW2AR-LV18QN88C8/I7 を対象とする。方向は FPGA から見たもの。

信号	方向	CST pin ／機能
`sys_clk / sys_reset`	I / I	4 / 88。reset は High で内部 reset_n を Low にする。
`UART_RXD / UART_TXD`	I / O	70 / 69
`PSCONE_SW1 / SW2`	I	86 / 79
`PSCONE_LED_OUT[0..5]`	O	15, 16, 17, 18, 19, 20
`SD_D3 / CLK / CMD / D0`	O/O/O/I	81 / 83 / 82 / 84（CS/SCK/MOSI/MISO）
LCD CS / RST / BL / DC	O	48 / 27 / 25 / 77
LCD SCK / SDI / SDO	O/O/I	41 / 49 / 28
I2S SCK / WS / LR / SD	O/O/O/I	29 / 30 / 31 / 26
SDRAM DQ / A / BA / DQM	I/O / O	幅 32 / 11 / 2 / 4。FPGA 内蔵 SDRAM 接続。

TP 端子群はトップに存在するが、端子宣言だけで touch 機能の動作を保証しない。I2C はトップの LV1 を SCL、LV2 を SDA に接続する。

電気的条件の扱い

CST の主要 I/O は LVCMOS33 で、一部に BANK_VCCIO=3.3 を指定する。これは I/O 制約であり、FPGA core 電源電圧や絶対最大定格ではない。元 TeX にあった 3.0〜3.6 V、133 MHz、60〜95 mA、温度範囲、架空の QFP/BGA 型番は根拠資料がないため仕様値として採用しない。電源、許容温度、駆動能力、Fmax は採用部品の資料、回路図、実機測定、対象 bitstream の timing report で確定する。

再現用ビルド例

以下は `PSC_RV32I` リポジトリ root から実行する例。

```
make -C PSC-ONE/software/os MODE=psc CPU_VERSION=v1 kernel_mem build/user.mem
make -C PSC-ONE/software/os fpga_fat32_boot
make -C PSC-ONE/software/os user_elf
cd PSC-ONE/hardware/sim
make -f Makefile.pscos simulate_PSCOS CPU_VERSION=v1 NPU_VERSION=legacy
```

Gowin EDA にはトップ、依存 RTL、PLL、ROM 初期値、対応 CST を登録する。CPU/NPU 各 1 版を選び、合成・配置配線・timing 確認後に bitstream を生成する。OS の `CPU_VERSION` 指定だけで FPGA 回路が切り替わるわけではない。

根拠：S02、S13、S14、S18。資料番号は最終頁を参照。

18　検証範囲、性能値、再現手順

既存記録と今回の検証を分ける

本書はソース照合と TeX/PDF 生成を目的とする。以下の CPU/NPU/OS 数値は保存済み資料の記録であり、今回の文書作成に伴う RTL 回帰・実機再測定ではない。

資料上の結果	読取り条件
CPU v1 ISA：49/49 PASS	`rv32ui-ma_data` を除く既存 suite。全 privileged 仕様の準拠認証ではない。
CPU v1 Yosys：1,876 cells	2026-09-26、`proc; opt; stat` 後の generic hierarchy。外部 cache/NPU 等を含まない。
CoreMark v1：80.879974	100 MHz 換算、0.808800/MHz、12.364 s。反復数との矛盾が未解決。
CoreMark v2：53.287861	同条件の記録、0.532879/MHz、18.766 s。
過去 TFLite Phase 4	小型 FC で CPU 966 µs、NPU 最速 tile 8 が 4,177 µs。現行長方形実装の性能値ではない。

CoreMark は 1-cycle 応答の CPU 単体 memory model で、SDRAM 待ち、cache miss、refill を含まない。v1 の score×time は約 1,000 だが資料の反復数は 500 とされているため、この値を無条件の製品性能として使わない。generic cell 数も FPGA LUT 数や DSP 数とは異なる。

CPU と SoC の回帰コマンド例

`PSC-ONE/hardware/sim` で実行し、CPU 別・NPU 別の構成を記録する。

```
make -f Makefile.riscv.sim simulate_RISCV_TESTS_PARALLEL CPU_VERSION=v1
make -f Makefile.cpu.core simulate_CPU_CORE CPU_VERSION=v1
make -f Makefile.cpu simulate_PSC_ONE_TESTS TEST_PROGRAM_LIST=basic CPU_VERSION=v1 NPU_VERSION=legacy
make -f Makefile.cpu simulate_PSC_ONE_TESTS TEST_PROGRAM_LIST=long CPU_VERSION=v1 NPU_VERSION=legacy
make -f Makefile.cpu simulate_PSC_ONE_TESTS TEST_PROGRAM_LIST=pulp CPU_VERSION=v1 NPU_VERSION=legacy
```

機能別に残す確認項目

- MMU：非恒等 map、4 KiB/4 MiB、R/W/X fault、PA 取得と VA の PC 保持。
- cache ／ NPU：byte store、dirty eviction、CPU/SA 競合、signed 境界、M 端数行、結果範囲。
- OS：ELF 形式不正・fault 復帰、FAT32 sector/cluster 境界、JPEG 形式・画素・方向。
- 推論：全中間値と出力一致、backend 切替、失敗時の結果無効化、AUTO 適用範囲。
- FPGA：対象 CST・PLL・全 SoC での timing、UART 起動、SD、LCD、I2C 実通信。

host モデル、単体 RTL、SoC RTL、配置配線、実機試験を別の証拠として扱う。局所 module の Fmax から全 SoC の最大動作周波数を推定しない。

根拠：S04、S06、S18、S19。資料番号は最終頁を参照。

19　出典、構成差分、文書管理

主要出典一覧

相対 path の基点は~/Program/PSC_RV32I/PSC-ONE/。コードは 2026-10-10 の作業ツリーを参照した。下表のディレクトリ指定は関連実装一式を示す。

番号	ファイル／調査対象
S01	README.md、README_JP.md、hardware/README.md
S02	hardware/rtl/tang20k/top/PSC_ONE_Chip.v、hardware/rtl/tang20k/timing/PSC_ONE_tang20k_cst.cst
S03	hardware/rtl/soc/soc_top/PSC_ONE_SOC.v
S04	docs/cpu.md、hardware/rtl/soc/cpu_v2_experimental/src/
S05	hardware/rtl/soc/cpu_v1/src/（wrapper、FetchUnit、Csr、MMU 等）
S06	docs/cpu_pulp_JP.md、docs/cpu_pulp_validation.md
S07	docs/cpu_mmu.md
S08	hardware/rtl/soc/cache/src/、hardware/rtl/tang20k/sdram_controller/
S09	hardware/rtl/soc/mmio/、hardware/rtl/soc/dma/PSC_ONE_DMA_axi.v
S10	hardware/rtl/soc/npu/README.md、hardware/rtl/soc/npu/src/
S11	hardware/rtl/soc/npu_v1/README.md、hardware/rtl/soc/npu_v2/README.md、hardware/rtl/soc/pfe/README.md
S12	software/os/src/api/synap_api.c、synap_api.h、sa_limits.h、sa_transfer.h（後 3 点は同ディレクトリ）
S13	hardware/rtl/soc/mmio/PSC_SDCARD/README.ja.md、board/PSC-ONE/README.md
S14	docs/psc_os.md、software/os/src/boot/、software/os/src/kernel/
S15	docs/psc_os_api.md、software/os/src/api/user.h、software/os/src/kernel/kernel_syscall.c
S16	software/os/README.md、software/os/src/api/jpeg_display.h
S17	software/os/tests/elf/README.md、software/os/elf_apps/README.md
S18	hardware/sim/Makefile.cpu、Makefile.pscos 等、docs/getting-started.md
S19	software/os/src/api/tflite/（API、runtime、Synap、Phase 4 記録）、software/micropython/ports/psc/modpsc.c

図版と再ビルド

掲載した 6 点の JPG は docs/images/ から本書の figures/ へ無変換で複製した。LuaLaTeX で直接読込み、原画像の縦横比を維持する。図中のロゴ・配線概念・写真は図示時点の記録として扱い、仕様値は現行コードを優先する。出典ファイルと図版の SHA-256 は同梱 PSC-ONE_SpecSheet.sources.txt に記録する。

```
make psc_doc
# latexmk -lualatex -interaction=nonstopmode -halt-on-error PSC-ONE_SpecSheet.tex
```

Rev.0.9 の整理内容

標準 NPU を legacy とし、CPU v1、CSR の M/K/N、C API 上限 64、I-cache 4 KiB ／ D-cache 1 KiB、write-allocate、AUTO backend 等を現行実装へ合わせた。旧版の電気定格と発注型番を削除し、根拠未確定の項目として整理した。元資料のライセンスと third-party notices は各配布元に従う。